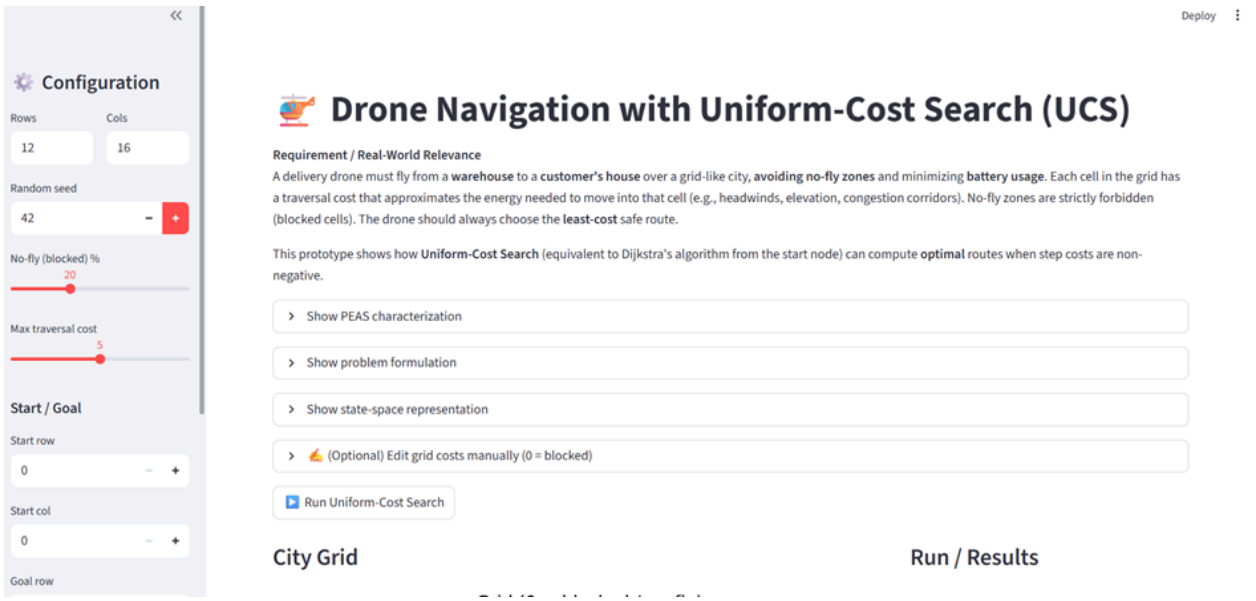
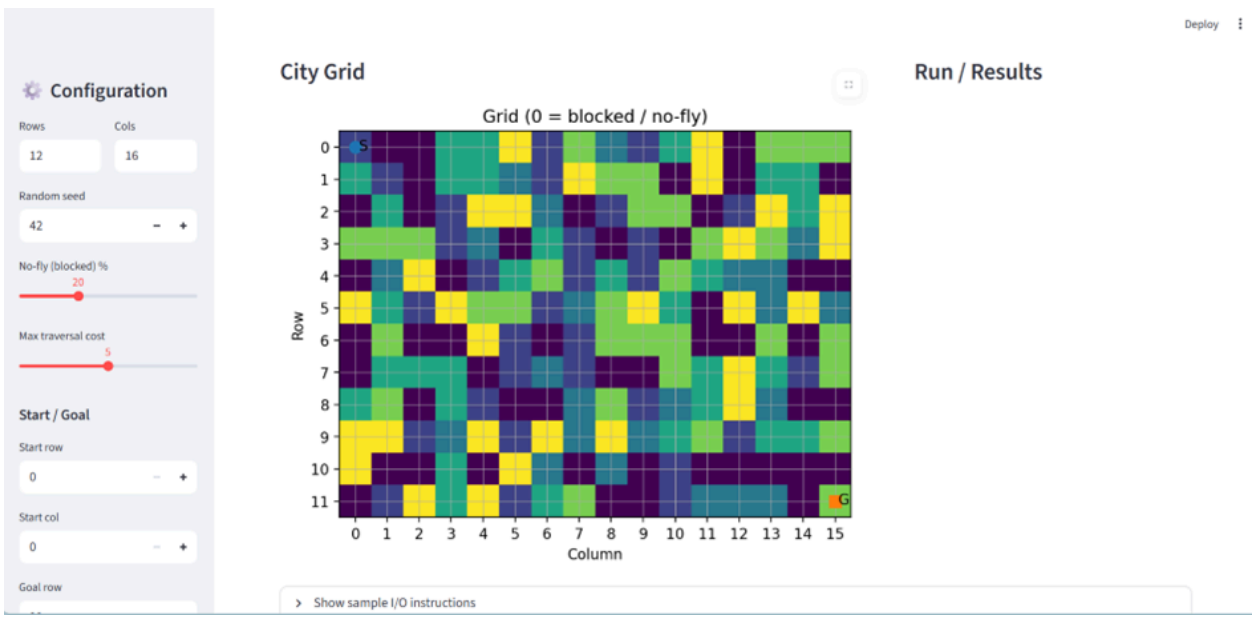
UIT2504 ARTIFICIAL INTELLIGENCE ASSIGNMENT-1

QUESTION 1





Deploy

Configuration

Rows: 12

Cols: 16

Random seed: 42

No-fly (blocked) %: 20

Max traversal cost: 5

Start / Goal

Start row: 0

Start col: 0

Goal row

Show sample I/O instructions

Sample Input/Output (click 'Load sample grid' then 'Run')

Sample grid (0 = blocked): preloaded 6×8 grid.
Start: (0,0) • **Goal:** (5,7)
Running UCS produces the **least-cost path** across the grid, avoiding the zero-cost (blocked) cells. The app displays:

- Total cost (battery)
- Nodes expanded
- Runtime
- The path overlaid on the grid

You can also **edit the grid** in the editor to create new scenarios.

Implementation notes

Notes & Assumptions

- Movement is 4-connected (no diagonals). Extending to 8-connected is trivial.
- The **step cost** is defined as the cost of **entering** the next cell.
- All costs must be **non-negative** to preserve UCS optimality.
- Goal test is performed when a node is **popped** from the priority queue.
- If a better path to a frontier node is found, its priority is updated by pushing a new entry (lazy decrease-key); stale entries are skipped on pop.

QUESTION 2

Deploy

Grid Setup

Grid Size (NxN): 8

Probability of Blocked Roads (%): 20

Traffic Multiplier (max): 3

Emergency Ambulance Routing using A* Search

This app simulates how an **ambulance** can find the **fastest route to a hospital** through a city grid with **traffic delays** and **blocked roads**, using *A Search**.

PEAS Characterization

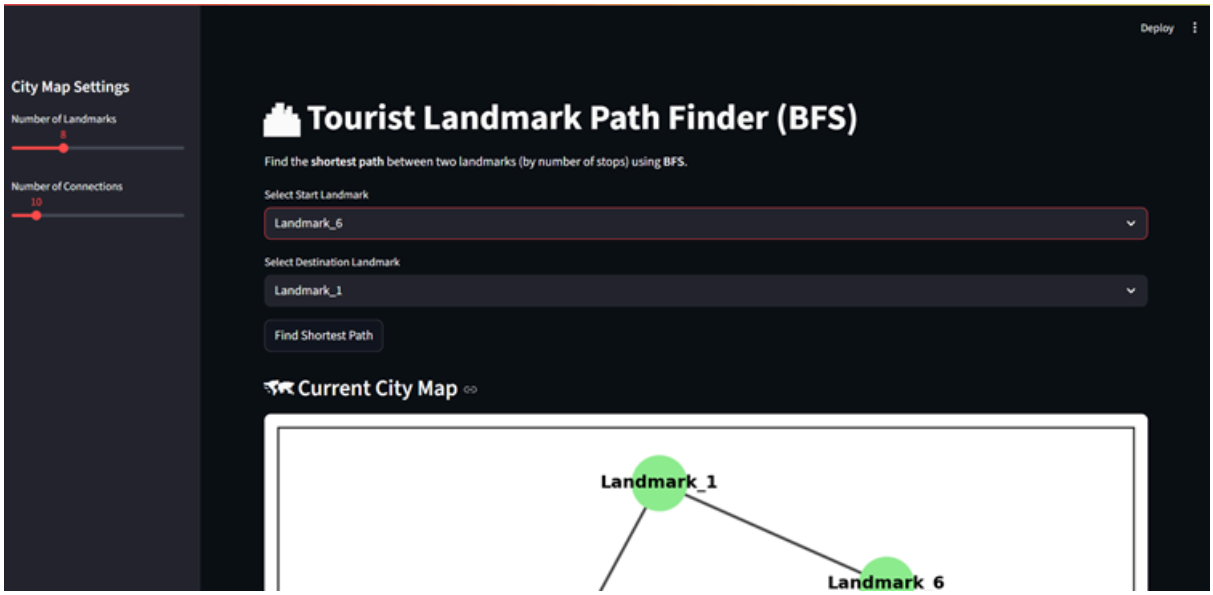
Problem Formulation

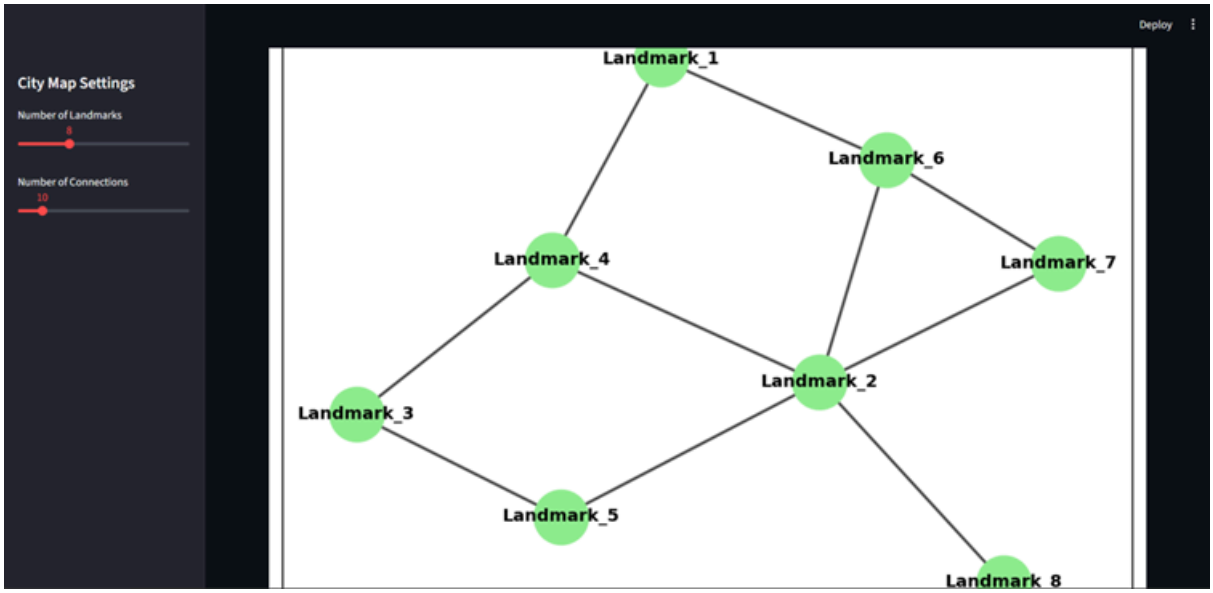
- **Initial State**: Ambulance starting position on the grid
- **Goal State**: Any hospital cell
- **Goal Test**: Check if ambulance reached hospital
- **Actions**: Move Up, Down, Left, Right (if not blocked)
- **Costs**: Travel time = traffic weight of cell
- **Heuristic**: Manhattan distance × min cell cost

Generated City Grid (0 = Blocked Road)

0	1	2	3	4	5	6	7
3	1	3	0	1	1	3	2
3	3	3	3	1	3	2	1
2	2	2	2	1	0	0	2

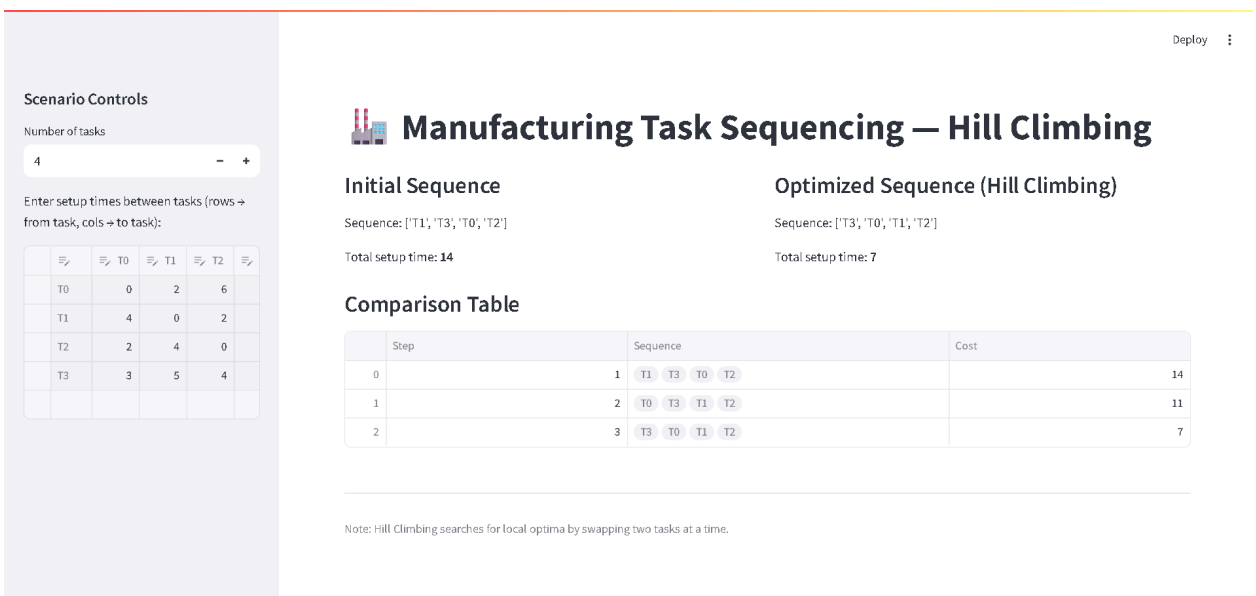
QUESTION 3


Deploy
City Map Settings
Number of Landmarks
8
Number of Connections
10
Tourist Landmark Path Finder (BFS)
Find the shortest path between two landmarks (by number of stops) using BFS.
Select Start Landmark
Landmark_6
Select Destination Landmark
Landmark_1
Find Shortest Path
Current City Map
Landmark_1
Landmark_6

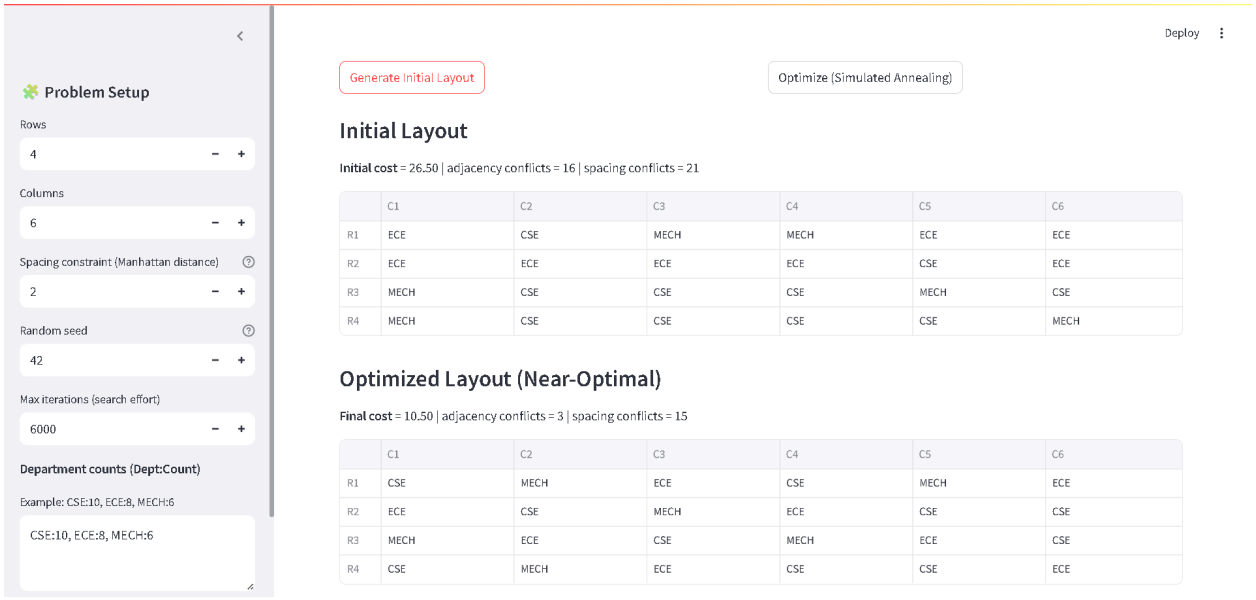

Deploy
City Map Settings
Number of Landmarks
8
Number of Connections
10
Landmark_1
Landmark_6
Landmark_4
Landmark_7
Landmark_2
Landmark_3
Landmark_5
Landmark_8

QUESTION 4

QUESTION 5:



QUESTION 6:



	C1	C2	C3	C4	C5	C6
R1	ECE	CSE	MECH	MECH	ECE	ECE
R2	ECE	ECE	ECE	ECE	CSE	ECE
R3	MECH	CSE	CSE	CSE	MECH	CSE
R4	MECH	CSE	CSE	CSE	CSE	MECH

	C1	C2	C3	C4	C5	C6
R1	CSE	MECH	ECE	CSE	MECH	ECE
R2	ECE	CSE	MECH	ECE	CSE	CSE
R3	MECH	ECE	CSE	MECH	ECE	CSE
R4	CSE	MECH	ECE	CSE	CSE	ECE

QUESTION 7

Scenario controls

Urgency weight (beta)

1.50

Show cost matrix

Uber Driver–Rider Assignment — A* (routing) + Local Search (matching)

Scenario description

Small synthetic city graph. Drivers and riders have node locations (A..G). A* computes full multi-node routes and ETAs (edge weights represent travel time/traffic). Matching first uses a greedy nearest-ETA assignment (respecting vehicle compatibility), then improves assignments with pairwise swap local search to reduce urgency-weighted ETA.

Drivers

- D1: node=A, vehicle=Sedan
- D2: node=B, vehicle=SUV
- D3: node=C, vehicle=Sedan

Riders

- R1: node=E, vehicle=SUV, urgency=3
- R2: node=D, vehicle=Sedan, urgency=2
- R3: node=B, vehicle=Sedan, urgency=1

Cost matrix (ETA) — rows: drivers, cols: riders

	0	1	2	3
0	Driver\Rider	R1	R2	R3
1	D1	INF	6.0	4.0
2	D2	3.0	INF	INF
3	D3	INF	4.0	6.0

Initial (Greedy) assignment

- D2 → R1 | ETA: 3.00 | Route: B → E
- D1 → R3 | ETA: 4.00 | Route: A → B
- D3 → R2 | ETA: 4.00 | Route: C → D

Initial urgency-weighted total cost = 42.50

Initial (Greedy) assignment

- D2 → R1 | ETA: 3.00 | Route: B → E
- D1 → R3 | ETA: 4.00 | Route: A → B
- D3 → R2 | ETA: 4.00 | Route: C → D

Initial urgency-weighted total cost = 42.50

Local search found no better pairwise swaps (greedy was already good).

Assignment comparison

Greedy

	driver	driver_loc	rider	rider_loc	urgency	eta	route
0	D2	B	R1	E	3	3	B E
1	D1	A	R3	B	1	4	A B
2	D3	C	R2	D	2	4	C D

Improved

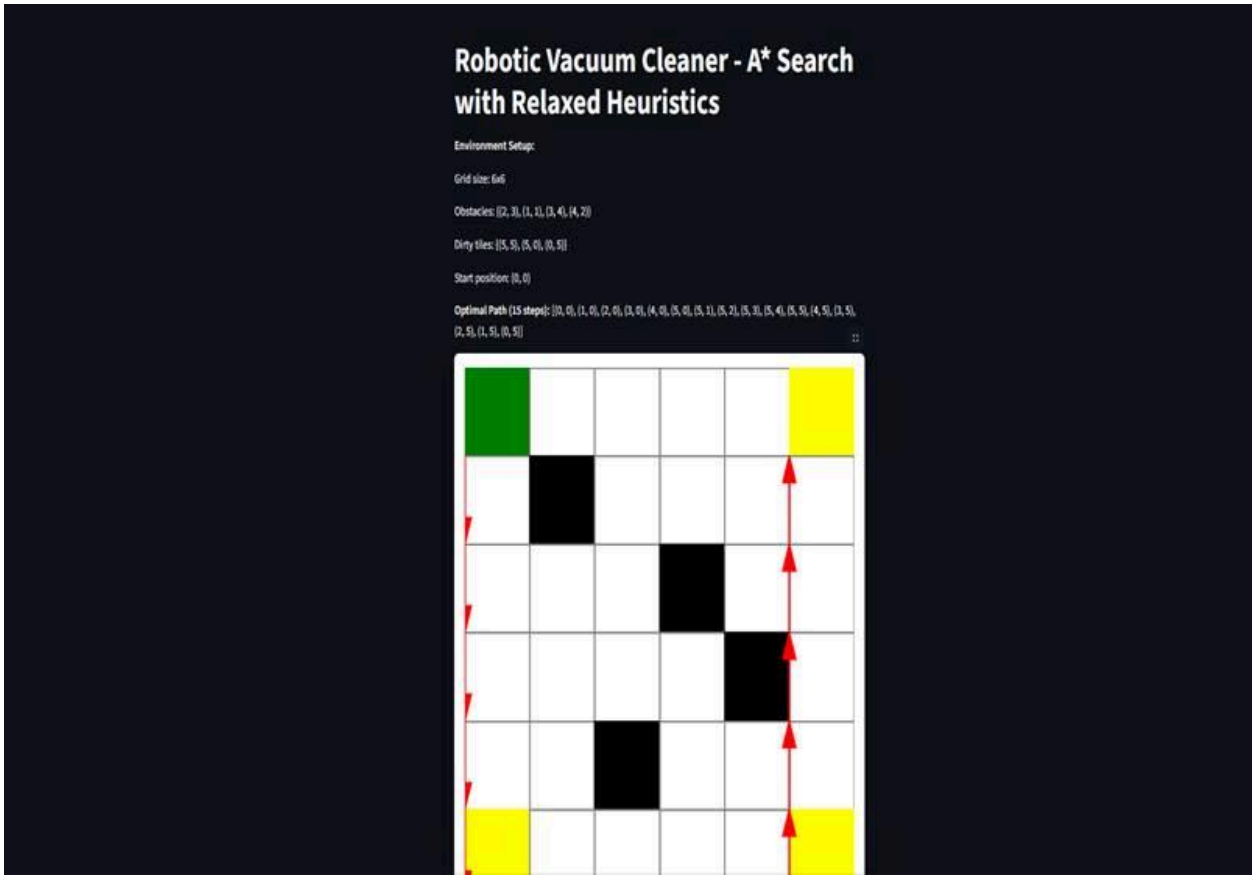
	driver	driver_loc	rider	rider_loc	urgency	eta	route
0	D2	B	R1	E	3	3	B E
1	D1	A	R3	B	1	4	A B
2	D3	C	R2	D	2	4	C D

Notes: A* produces full multi-node paths (shown under 'route'). Local search here is a simple pairwise-swap improvement — in production you'd add swaps, relocations, capacity constraints, and time windows.

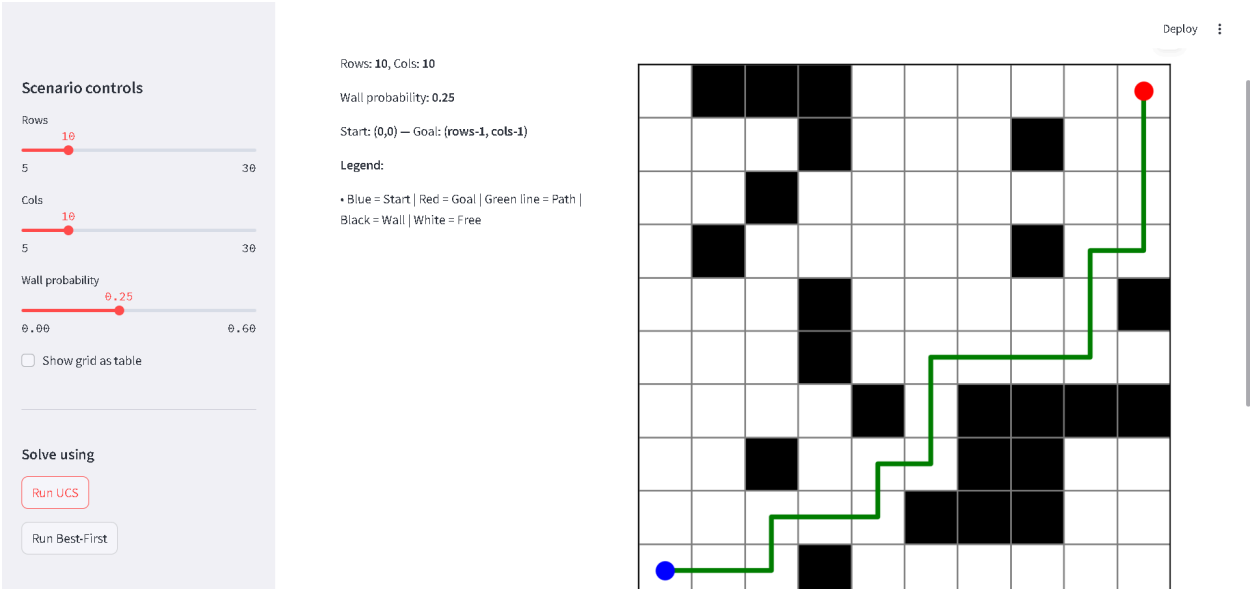
QUESTION 8:

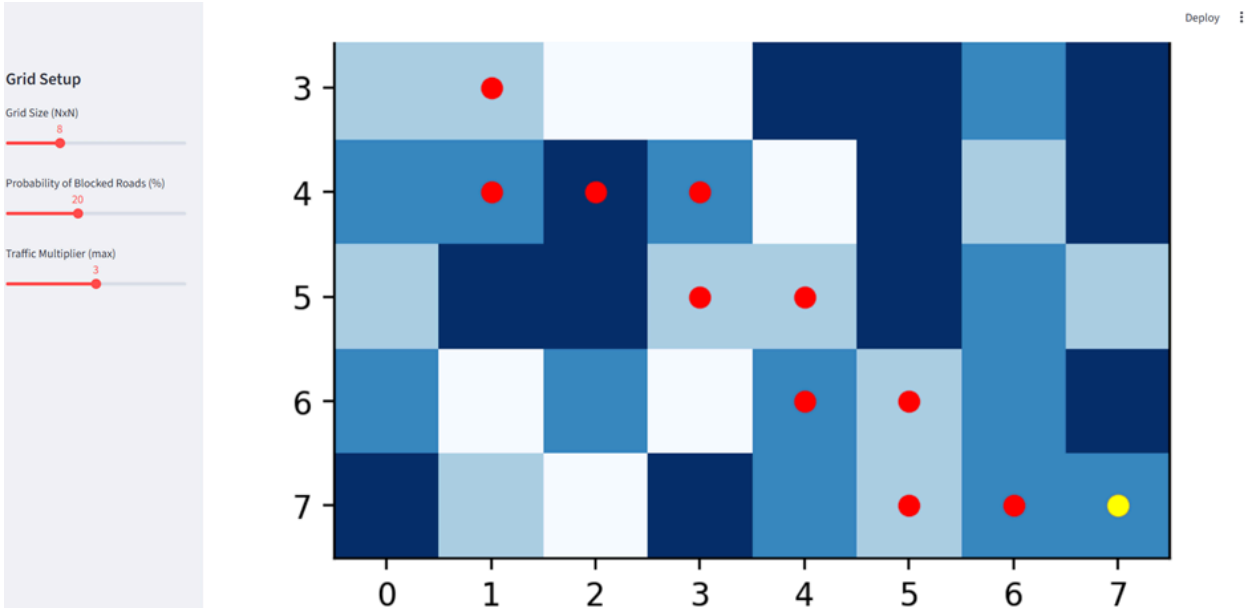
Deploy
Controls
Generate Random Grid
Solve with BFS
Solve with DFS
Solve with IDDFS
Pathfinding Visualizer

QUESTION 9


Robotic Vacuum Cleaner - A* Search with Relaxed Heuristics
Environment Setup:
Grid size: 6x6
Obstacles: [(2, 3), (1, 1), (3, 4), (4, 2)]
Dirty tiles: [(5, 5), (5, 0), (0, 5)]
Start position: (0, 0)
Optimal Path (15 steps): [(0, 0), (1, 0), (2, 0), (3, 0), (4, 0), (5, 0), (5, 1), (5, 2), (5, 3), (5, 4), (5, 5), (4, 5), (3, 5), (2, 5), (1, 5), (0, 5)]

QUESTION 10:


Deploy
Scenario controls
Rows
10
5
30
Cols
10
5
30
Wall probability
0.25
0.00
0.60
Show grid as table
Solve using
Run UCS
Run Best-First
Rows: 10, Cols: 10
Wall probability: 0.25
Start: (0,0) — Goal: (rows-1, cols-1)
Legend:
• Blue = Start | Red = Goal | Green line = Path | Black = Wall | White = Free


Deploy
Grid Setup
Grid Size (NxN)
8
Probability of Blocked Roads (%)
20
Traffic Multiplier (max)
3
3
4
5
6
7
0
1
2
3
4
5
6
7